Course Overview

Introduction to Robotics

Prof. Anis Koubaa

EE 414 — Introduction to Robotics
College of Engineering
Alfaisal University

Fall 2026 — Week 1, Lecture A

Who is teaching this

Prof. Anis Koubaa

- Professor of Computer Science, Alfaisal University
- Director, Robotics and Internet-of-Things (RIOTU) Lab
- Research: mobile robots, drones, deep learning, IoT

On this subject

Author of best-selling online courses on ROS, and of the Springer *Robot Operating System* book series.

Why this matters to you

The material in this course is the material I teach to practising engineers. It is not a simplified classroom version — which is the reason it is demanding, and the reason it is worth the semester.

What this lecture does

Roadmap

What EE 414 asks you to learn, why it is built the way it is, how you will be assessed, and what you must do before the next class.

What this course is about

Warehouse robots, at scale

45 s — an autonomous mobile robot fleet moving stock in a live warehouse

The largest deployed fleet of *exactly* the software stack in this course.

Where people should not go

45 s — a legged or wheeled robot inspecting an industrial plant

No map, no GPS, nobody nearby. Week 10 is about the first of those.

A robot changing its mind

30 s — a mobile robot re-planning around a person who steps into its path

You will configure this exact behaviour in Week 11.

Built here

45 s — research robots and drones from the RIOTU Lab

Every one of these runs on what you are about to learn.

The course in one sentence

EE 414 teaches you to make a machine **act, on its own, in the physical world.**

ROS 2 is the framework we write those machines in. It is the tool, not the subject.

From the course specification

"This course introduces the foundations of robotics — modelling, control, perception, localization, mapping and motion planning — and develops them through practical software implementation using the Robot Operating System 2 (ROS 2)."

Two courses you could take. This is the second one.

A robotics theory course

You derive the kinematics.
You solve for the trajectory.
You are examined on the derivation.

You graduate having **never made a robot move**.

This course

You derive the kinematics on Sunday.
On Tuesday the robot drives with it.
When it does not drive, you find out why.

You graduate having **built the thing**.

The mathematics is not reduced. It is *finished* — an equation is finished when something moves because of it.

Where you end up

By Week 11 you will have a robot that does this, and you will have written every part of it:

builds a map of a room it has never seen

works out where it is in that map

plans a path to a goal you click

drives there, around what is in the way

In the field

The same stack — ROS 2, SLAM, Nav2 — runs on warehouse robots and inspection rovers today. Configuring it is a real first week of work.

Why robotics is hard, and what we do about it

Three reasons robotics is harder than the software you have written

The difficulty	Why it bites	What this course does
The world is uncertain	Your sensor reports 2.31 m. The wall is at 2.28 m. Neither number is wrong.	Estimation is taught as a first-class topic (Weeks 9–10), not an appendix.
Everything runs at once	Sensing, planning and control do not take turns. They all run, always, at different rates.	ROS 2 from Week 1, so concurrency is the normal case and not a shock.
There is no undo	A wrong loop prints wrong output. A wrong velocity command hits a wall.	Simulation first, every week. Hardware only once the code has earned it.

The gap this course exists to close

What you can already do

- Write a Python program that computes an answer
- Design a feedback controller on paper (EE 306)
- Analyse a system in the frequency domain

What a robot needs

- Ten programs running at once, talking to each other
- A controller whose input arrives late, noisy, and sometimes not at all
- A decision every 50 ms, forever, with no human watching

The point

EE 306 gave you the controller. This course gives you everything between the controller and the wheels — and that "everything" is where robots actually fail.

The four course learning outcomes

	By the end of this course you can...	Bloom level
CLO 1 EXPLAIN	Explain robot system architectures and the ROS 2 computation graph.	Understand
CLO 2 ANALYZE	Analyze robot kinematics, motion control and state estimation problems.	Analyze
CLO 3 DEVELOP	Develop ROS 2 software for robot perception, localization and navigation.	Create
CLO 4 EVALUATE	Evaluate ethical and safety implications of autonomous robots within a team.	Evaluate

Four outcomes. One verb each. Short enough that you can recall them without opening the document — and every exam question you will sit maps to exactly one of them.

How the course works

The weekly shape

Lecture A — 1.5 h
the idea, the mathematics, the derivation

$\xrightarrow{\text{same week}}$

Lecture B — 1.5 h
the same idea, running, in ROS 2

- **Bring a laptop to Lecture B.** It is not a demonstration you watch.
- Every B session ends with something that **runs** — your node printing, your robot moving, your frame appearing in RViz2.
- If you leave a B session with a half-typed file, tell me before you leave. That is a failure of the session, not of you.

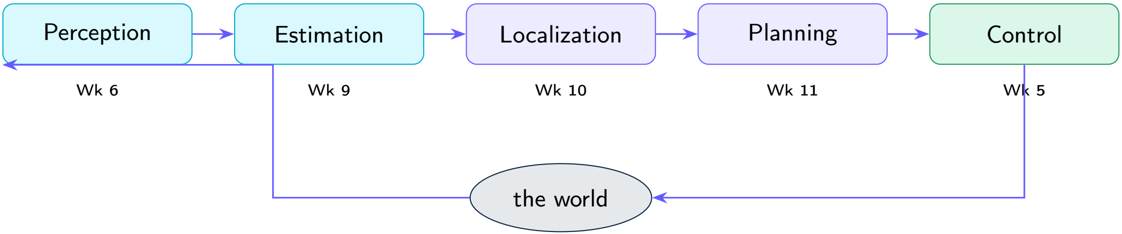
The semester: 11 taught weeks, then 4

Wk	Topic	Due
1	Robotics and the ROS 2 ecosystem	—
2	ROS 2 computation graph	Quiz 1
3	Topics, services, actions	Asgn 1
4	Differential-drive kinematics	Quiz 2
5	Feedback control for motion	Asgn 2
6	Sensors, actuators, LiDAR	**MT I**
7	Transformations and TF2	Quiz 3
8	Robot modelling: URDF, Gazebo	Asgn 3

Wk	Topic	Due
9	Probabilistic state estimation	Quiz 4
10	Localization, mapping (SLAM)	Asgn 4
11	Motion planning, Nav2	Quiz 5
12	Integration + advanced topic	**MT II**
13	Robot ethics and safety	Review
14	Review and project demos	**Project**
15	—	**Final**

Weeks 12–15 carry no new required material. A week lost to a holiday or a university event is absorbed **without dropping a topic**.

Every week is one block of one robot



Weeks 2–3 build the *plumbing* that lets these blocks talk. Weeks 4 and 7–8 build the *model* they all share. Nothing in this course is a standalone topic — by Week 12 every block is running at the same time, on the same robot.

Course information

Practical information

Instructor	Prof. Anis Koubaa — `akoubaa@alfaisal.edu`
Office hours	Announced in the syllabus, and by appointment
Credits	3 (3-0-0) — two 1.5-hour sessions each week
Prerequisite	EE 306 Control and Feedback Systems Design
Assumed	Python, and enough Linux to move around a terminal
LMS	`elearning.alfaisal.edu` — announcements, grades, submissions
Course repo	Slides, starter packages, lab sheets. Pull before every class.

If your Linux is weak

Say so in Week 1, not Week 6. The setup guide has a short primer, and the Week 2 session assumes you have worked through it. Nobody has ever failed this course on Linux — people fail it by hiding that they were lost in Week 2.

The stack, pinned

Middleware	ROS 2 Jazzy
Operating system	Ubuntu 24.04 LTS
Simulator	Gazebo Harmonic
Language	Python 3.12 (`rclpy`)
Visualisation	RViz2
Robot	TurtleBot3 — in simulation, and on hardware where available

- **Everything is done in simulation first.** Gazebo is not a lesser version of the course; it is where the work happens. Hardware is a bonus, not a dependency.
- **Python only.** Production robotics uses C++ as well, and we will say where and why — but nothing in this course is assessed in it.

What this course does *not* cover

Forty-five contact hours buys one complete pipeline, taught properly. It does not buy everything. Know this in Week 1, not Week 9:

Not here	**Where to get it**
Computer vision, OpenCV	A dedicated vision course; the perception in EE 414 is LiDAR
Aerial robots, drones	Same foundations, different dynamics — take this course first
Robotic manipulation	One optional session in Week 12, if the class chooses it
C++ for ROS 2	Production robotics uses it; this course assesses Python only
ROS 1, `catkin`, `roscore`	Superseded. We teach ROS 2 throughout.

Ask me for reading on any of these. **What you get instead** is one robot that maps a room it has never seen and drives across it — built by you, end to end.

Books — and the one that is not a book

Essential

- P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in Python*, 3rd ed., Springer, 2023.
- The **official ROS 2 documentation**, `docs.ros.org`. Treated as a primary source, and **examinable**.

Supportive — for the weeks that need depth

- Siegwart, Nourbakhsh, Scaramuzza, *Introduction to Autonomous Mobile Robots* — kinematics, sensors
- Thrun, Burgard, Fox, *Probabilistic Robotics* — estimation, SLAM
- Lynch & Park, *Modern Robotics* — rigid-body transformations

In the field

Reading documentation *is* the professional skill here — CLO 1 names it. In this field a textbook is three years behind the tool you are being paid to use.

Assessment

How the 100% is made

Component	%	When
Attendance and participation	5	throughout
ROS 2 assignments (4)	15	Wk 3, 5, 8, 10
Quizzes (5, short)	5	Wk 2, 4, 7, 9, 11
Midterm Exam I	15	Week 6
Midterm Exam II	15	Week 12
Course project	20	Weeks 13–14
Final Exam	25	Exam week
Total	**100**	

Two mercies

The **lowest assignment** is dropped.
The **lowest quiz** is dropped.

One bad week does not follow you to December.

One warning

There are **no bonus marks**. Plan on the 100 that exists.

What a written exam looks like for a software course

A robotics exam that only asks you to derive equations does not measure this course. Every exam here is built to a fixed blueprint:

Section	What you do	Weight
Recall	Definitions, ROS 2 vocabulary	$\leq$10%
Explain	Why a mechanism exists, how the architecture fits	20%
Analyze	Kinematics, control, estimation, planning — worked by hand	30%
Read code	Given a node: what does it do, what does it publish?	20%
Write code	Complete a node, a callback, a launch entry, on paper	20%

40% of every exam is code, on paper. If you have only ever copied commands from a tutorial, that 40% is where it shows. This is deliberate.

Assignments and the project

Four ROS 2 assignments (Weeks 3, 5, 8, 10). Each extends what the Lecture B session started. Marked on functionality 50, ROS 2 idiom and package structure 20, code quality 20, report or demo 10.

The project — teams of 3, Weeks 13–14, 20% of the course.

- A complete autonomous behaviour: warehouse delivery, coverage cleaning, frontier exploration, person following — or your own proposal.
- Deliverables: a Git repository, a live demo, a 6-page report, a 10-minute talk.
- **Proposal due Week 5.** Teams form in Week 4. Start thinking now.

The individual question

At the demo, each member is asked about a part of the system they did *not* write. A team that cannot explain its own robot has not met CLO 3, whatever the robot does.

Policies

Attendance, integrity, and the rules that matter

Attendance. University policy, strictly applied. Attendance is taken in the first minutes. Missing a Lecture B session is expensive in a way that missing a lecture is not — the environment moves forward and you do not.

Academic integrity. Submitting another person's work as your own — copied code, a repository forked without attribution, a paraphrased report — brings course failure and possible suspension, per the university policy.

Open-source code. Robotics runs on other people's packages, and using them is correct engineering. **Attribute it.** Name the package, the version and the licence in your report. Using `slam_toolbox` is expected; presenting it as yours is plagiarism. We spend Week 13 on exactly this line.

Using AI coding tools

Permitted in assignments and the project. **Not** permitted in examinations.

Two conditions, and they are not negotiable:

1. **Disclose.** Say what you used and for what, in the report.
2. **Defend.** Be able to explain and justify every line you submit, on the spot.

In the field

An assistant will happily give you ROS 1 code, or code for a distribution three versions old, and it will look right. Debugging that is a skill this course teaches — and the reason condition 2 exists. The exam is on paper for the same reason.

Getting set up

Before the next class

Everything from Week 2 onward assumes a working ROS 2 Jazzy installation. Budget **60–90 minutes**. Start tonight, not the night before.

Route	For	Note
A — Native Ubuntu 24.04 LTS	A spare machine or dual boot	Best performance
B — WSL2 on Windows 11	Most of you	GUI apps work via WSLg
C — Docker	macOS, incl. Apple Silicon	Gazebo runs slower
D — Lab machines	Fallback	Pre-imaged; nothing persists

The guide is in `setup/` in the course repository, with a script that checks your installation and prints a pass/fail table. **Run it. Bring the output to Week 2.**

One setting that will save the whole room

ROS 2 nodes find each other automatically over the network. In a shared lab, that means **your robot will be driven by the person sitting behind you** — and neither of you will understand why.

```
$ echo "export ROS_DOMAIN_ID=17" >> ~/.bashrc
$ source ~/.bashrc
```

You will be assigned a number this week. Put it in your `.bashrc` today.

Under the hood

The domain ID partitions the discovery traffic: nodes only see nodes on the same ID. It is the first thing to check when a node "cannot see" a topic that is obviously being published.

Your checklist for this week

1. Clone the course repository, and read `setup/README.md`.
2. Install ROS 2 Jazzy by the route that fits your machine.
3. Run the setup check script — every row must say `PASS`.
4. Set your assigned `ROS_DOMAIN_ID`.
5. Skim the ROS 2 "Concepts" page. You will not understand all of it. Skim it anyway — Lecture 2A is much easier when the vocabulary is not brand new.
6. Start thinking about who you want in your project team.

If you get stuck

Post on the LMS forum, not by email — twenty people hit the same installation error, and the first person to post it saves the other nineteen.

Next: Lecture 1B

What *is* a robot — and why does robot software need a framework at all?

We will look at:

- What separates a robot from a very good machine
- The sense–plan–act loop, and the four blocks every autonomous robot has
- Three hard truths about the physical world that shape all of robotics
- Why nobody writes robot software as one program any more — and what ROS 2 does about it

Questions now, or `akoubaa@alfaisal.edu`.